```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical # For one-hot encoding labels
from tqdm import tqdm # For progress bars
import time
```

1. Forward Diffusion Process on MNIST Digits.

```
BATCH_SIZE = 128
T = 200          # Total diffusion steps
```

```
beta_start = 1e-4
beta_end = 0.02
beta = tf.linspace(beta_start, beta_end, T, name="beta_schedule")
beta = tf.cast(beta, dtype=tf.float32)
```

```
alpha = 1.0 - beta
alpha_hat = tf.math.cumprod(alpha, name="alpha_hat_schedule")
```

```
# Loading and preprocessing MNIST data
print("\nLoading and preprocessing MNIST data...")
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
```

```
Loading and preprocessing MNIST data...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
x_train = (x_train.astype(np.float32) / 255.0) * 2.0 - 1.0
x_train = np.expand_dims(x_train, axis=-1)
print(f"x_train shape: {x_train.shape}")
```

```
x_train shape: (60000, 28, 28, 1)
```

```
def q_sample(x_start, t, noise=None):
    if noise is None:
        noise = tf.random.normal(shape=tf.shape(x_start))
    if isinstance(t, int):
        t_val = t
        t = tf.fill(dims=(tf.shape(x_start)[0],), value=t_val)
    elif not isinstance(t, tf.Tensor):
        t = tf.convert_to_tensor(t, dtype=tf.int32)

    sqrt_alpha_hat_t = tf.gather(tf.sqrt(alpha_hat), t)
    sqrt_one_minus_alpha_hat_t = tf.gather(tf.sqrt(1.0 - alpha_hat), t)

    sqrt_alpha_hat_t = tf.reshape(sqrt_alpha_hat_t, [-1, 1, 1, 1])
    sqrt_one_minus_alpha_hat_t = tf.reshape(sqrt_one_minus_alpha_hat_t, [-1, 1, 1, 1])

    # Calculate the noisy image x_t
    return sqrt_alpha_hat_t * x_start + sqrt_one_minus_alpha_hat_t * noise

print("q_sample function defined.")
```

```
q_sample function defined.
```

```
# Show visual samples at step t = [1, 50, 100, 150, 200]
# So, t=1 (step 1) is index 0, t=50 is index 49, t=200 is index 199.
```

```
timesteps_to_show = [0, 49, 99, 149, 199]
```

```
num_sample_digits = 5
```

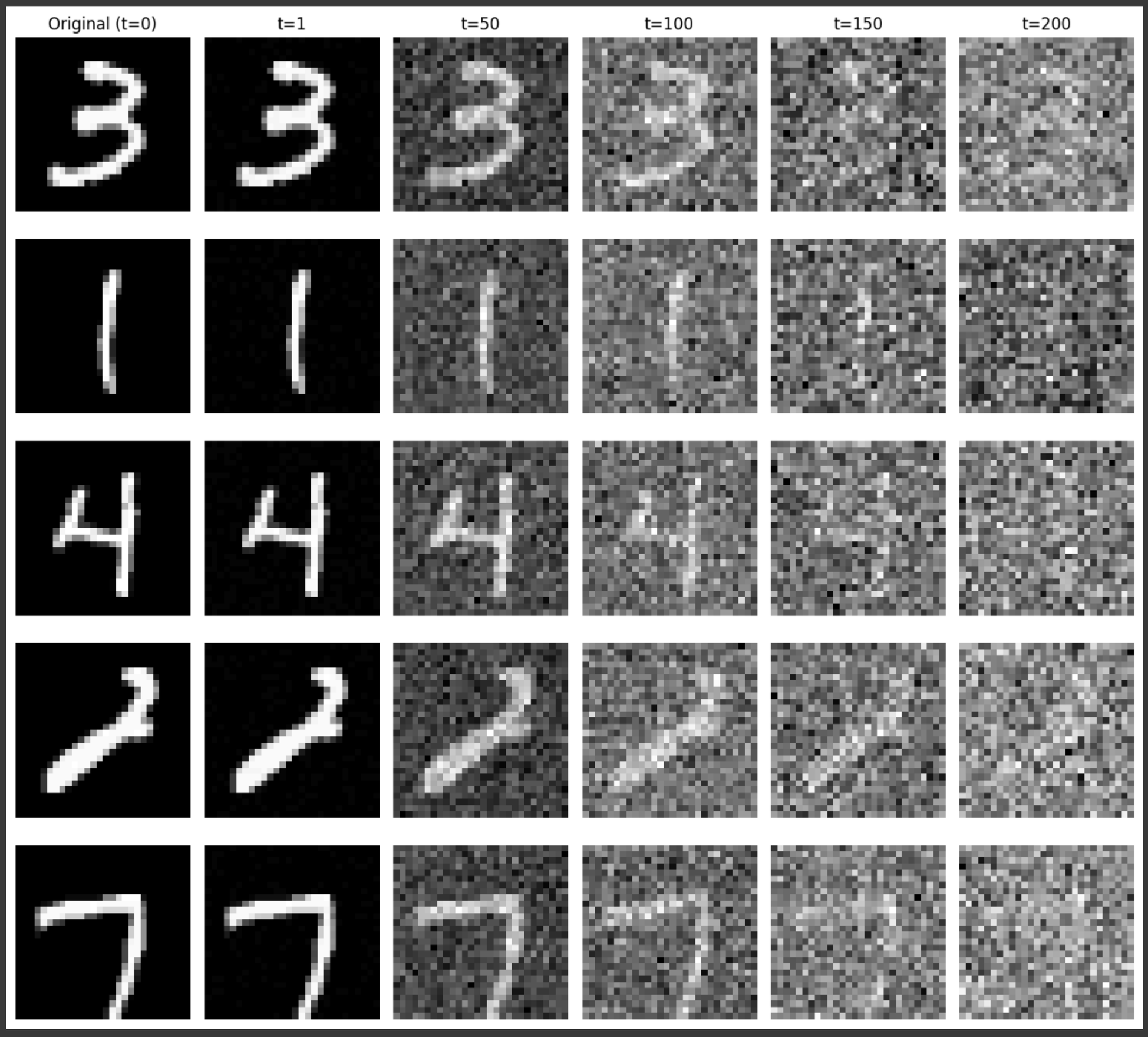
```
sample_digits_indices = np.random.choice(len(x_train), num_sample_digits, replace=False)
```

```
sample_digits = tf.convert_to_tensor(x_train[sample_digits_indices], dtype=tf.float32)
```

```
print(f"\nVisualizing forward diffusion for {num_sample_digits} sample digits at timesteps: {np.array(timesteps_to_show)+1}...")
```

```
Visualizing forward diffusion for 5 sample digits at timesteps: [  1  50 100 150 200]...
```

```
plt.figure(figsize=(12, num_sample_digits * 2.2)) # Adjusted figure size
for i, digit_idx in enumerate(range(num_sample_digits)):
    original_image = sample_digits[digit_idx]

    # Plot original image
    plt.subplot(num_sample_digits, len(timesteps_to_show) + 1, i * (len(timesteps_to_show) + 1) + 1)
    plt.imshow((original_image.numpy() + 1.0) / 2.0, cmap='gray') # Scale back to [0,1] for display
    if i == 0:
        plt.title("Original (t=0)")
    plt.axis('off')

    # Plot noisy images at specified timesteps
    for j, t_val in enumerate(timesteps_to_show):
        x_start_batch = tf.expand_dims(original_image, axis=0)
        noisy_image = q_sample(x_start_batch, t_val) # t_val is 0-indexed

        plt.subplot(num_sample_digits, len(timesteps_to_show) + 1, i * (len(timesteps_to_show) + 1) + j + 2)
        plt.imshow((noisy_image[0].numpy() + 1.0) / 2.0, cmap='gray') # Display first image in batch
        if i == 0:
            plt.title(f"t={t_val+1}") # Display 1-indexed step number
        plt.axis('off')

plt.tight_layout()
plt.show()
```

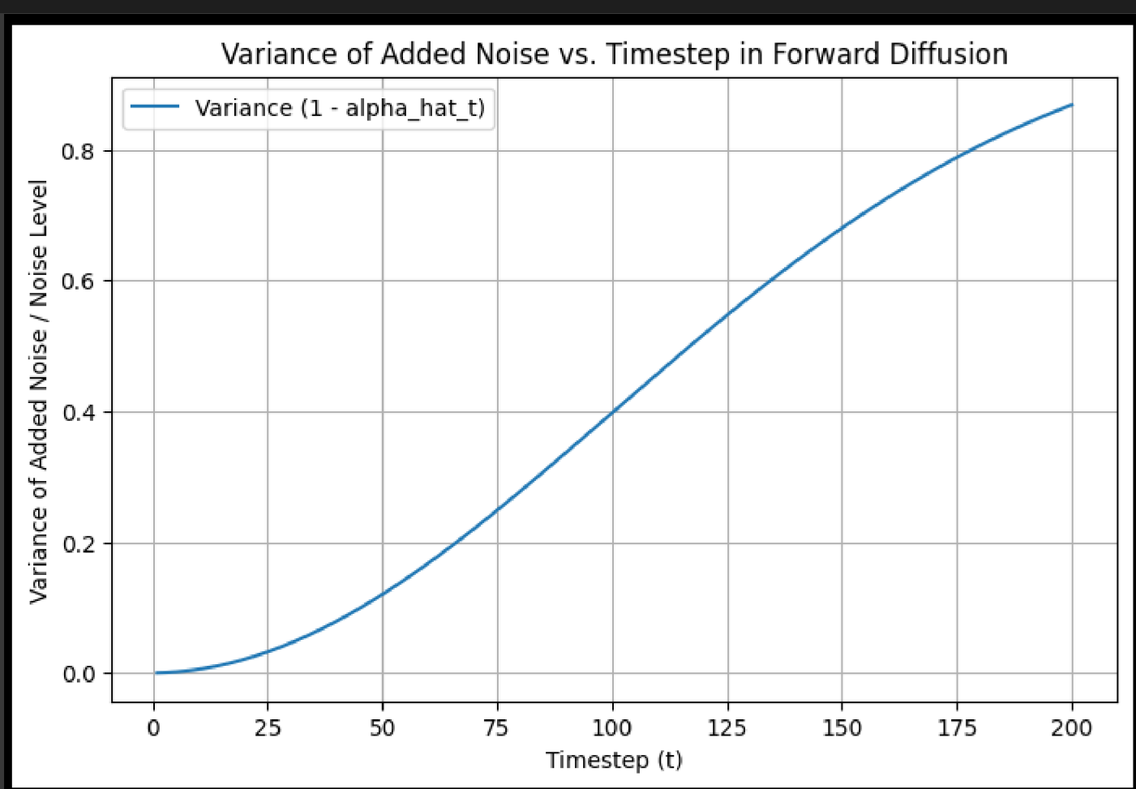



```
# Plotting variance/noise level vs timestep
```

```
variance_schedule = 1.0 - alpha_hat
```

```
timesteps_for_plot = np.arange(T)
```

```
plt.figure(figsize=(8, 5))
plt.plot(timesteps_for_plot + 1, variance_schedule.numpy(), label='Variance (1 - alpha_hat_t)')
plt.xlabel('Timestep (t)')
plt.ylabel('Variance of Added Noise / Noise Level')
plt.title('Variance of Added Noise vs. Timestep in Forward Diffusion')
plt.grid(True)
plt.legend()
plt.show()
```




```
print(f"Variance at t=1 (index 0): {variance_schedule[0]:.4f}")
```

```
Variance at t=1 (index 0): 0.0001
```

```
print(f"Variance at t={T} (index {T-1}): {variance_schedule[T-1]:.4f}")
```

```
Variance at t=200 (index 199): 0.8678
```

2. Backward Denoising Step with a Simple U-Net or Linear Model

```
# Diffusion Parameters
T = 200
beta_start = 1e-4
```

```
beta_end = 0.02
beta = tf.linspace(beta_start, beta_end, T)
beta = tf.cast(beta, dtype=tf.float32)
alpha = 1.0 - beta
alpha_hat = tf.math.cumprod(alpha)
```

```
# Loading and preprocessing MNIST data
print("\nLoading and preprocessing MNIST data...")
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train.astype(np.float32) / 255.0) * 2.0 - 1.0  # Scale to [-1, 1]
x_train = np.expand_dims(x_train, axis=-1)
print(f"x_train shape: {x_train.shape}")
```

```
Loading and preprocessing MNIST data...
x_train shape: (60000, 28, 28, 1)
```

```
BATCH_SIZE_Q2 = 128
train_dataset_q2 = tf.data.Dataset.from_tensor_slices(x_train).shuffle(10000).batch(BATCH_SIZE_Q2)
print("MNIST dataset prepared.")
```

```
MNIST dataset prepared.
```

```
# Forward Diffusion Process Function (q_sample from Q1 )
def q_sample(x_start, t, noise=None):
    if noise is None:
        noise = tf.random.normal(shape=tf.shape(x_start))
    if isinstance(t, int):
        t_val = t
        t = tf.fill(dims=(tf.shape(x_start)[0],), value=t_val)
    elif not isinstance(t, tf.Tensor):
        t = tf.convert_to_tensor(t, dtype=tf.int32)

    sqrt_alpha_hat_t = tf.gather(tf.sqrt(alpha_hat), t)
    sqrt_one_minus_alpha_hat_t = tf.gather(tf.sqrt(1.0 - alpha_hat), t)
    sqrt_alpha_hat_t = tf.reshape(sqrt_alpha_hat_t, [-1, 1, 1, 1])
    sqrt_one_minus_alpha_hat_t = tf.reshape(sqrt_one_minus_alpha_hat_t, [-1, 1, 1, 1])
    return sqrt_alpha_hat_t * x_start + sqrt_one_minus_alpha_hat_t * noise
```

```
# Building a minimal denoising model (2-layer MLP using only tensor ops)
```

```
img_height, img_width, channels = 28, 28, 1
```

```
img_flat_size = img_height * img_width * channels
```

```
time_embedding_dim = 32
```

```
# Layer 1 parameters
hidden_size1 = 256
```

```
# Layer 2 parameters (output layer)
# Output size must be img_flat_size to predict noise of the same shape as image
# Initialize weights and biases as tf.Variable
W_time_emb = tf.Variable(tf.random.normal([T, time_embedding_dim]) * 0.02, name="W_time_emb")
```

```
# MLP Layer 1
W1 = tf.Variable(tf.random.normal([img_flat_size + time_embedding_dim, hidden_size1]) * 0.02, name="W1")
b1 = tf.Variable(tf.zeros([hidden_size1]), name="b1")
```

```
# MLP Layer 2 (Output layer)
W2 = tf.Variable(tf.random.normal([hidden_size1, img_flat_size]) * 0.02, name="W2")
b2 = tf.Variable(tf.zeros([img_flat_size]), name="b2")
```

```
model_params = [W_time_emb, W1, b1, W2, b2]
print("Manual 2-layer MLP (for noise prediction) parameters initialized.")
```

```
Manual 2-layer MLP (for noise prediction) parameters initialized.
```

```
# Manual ReLU activation
def manual_relu(x):
    return tf.maximum(x, 0.0)
```

```
def manual_denoise_model_forward(noisy_images_input, timesteps_input):

    # Flatten noisy images
    batch_s = tf.shape(noisy_images_input)[0]
    x_flat = tf.reshape(noisy_images_input, [batch_s, -1])

    # Timestep embedding using tf.gather
    t_embedded = tf.gather(W_time_emb, timesteps_input)

    # Concatenate flattened image and timestep embedding
    combined_input = tf.concat([x_flat, t_embedded], axis=1)
    # Layer 1
    h1_pre_activation = tf.matmul(combined_input, W1) + b1
    h1_activated = manual_relu(h1_pre_activation)

    # Layer 2 (Output - predicted noise, flat)
    pred_noise_flat = tf.matmul(h1_activated, W2) + b2

    # Reshape predicted noise to image shape
    pred_noise_img_shape = tf.reshape(pred_noise_flat, tf.shape(noisy_images_input))
    return pred_noise_img_shape

print("Manual model forward pass function defined.")
```

```
Manual model forward pass function defined.
```

```
# Using Mean Squared Error loss
def manual_mse_loss(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
```

```
# Optimizer (SGD)
learning_rate_q2 = 1e-3

def manual_sgd_step(params_list, grads_list, lr):
    for param, grad in zip(params_list, grads_list):
```

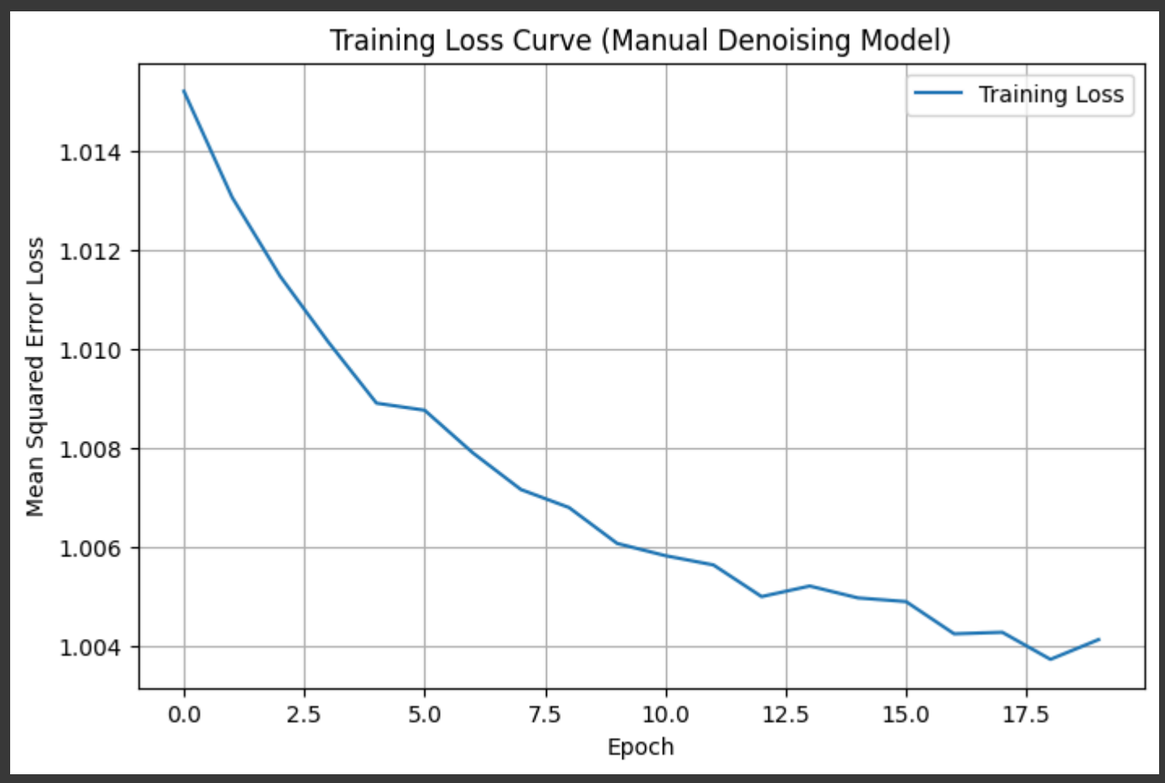
```
        if grad is not None:
            if isinstance(grad, tf.IndexedSlices):
                dense_grad = tf.convert_to_tensor(grad)
                param.assign_sub(lr * dense_grad)
            else:
                param.assign_sub(lr * grad)
```

```
#Train to reverse the noise (predict noise)
# Training Step Function
@tf.function
def train_step_manual_model(x_clean_batch):
    current_batch_size = tf.shape(x_clean_batch)[0]
    # Sample random timesteps for each image in the batch
    t_batch = tf.random.uniform(minval=0, maxval=T, shape=[current_batch_size], dtype=tf.int32)
    # Generate noise to be added
    actual_noise_added = tf.random.normal(shape=tf.shape(x_clean_batch))
    # Create noisy images
    x_t_noisy = q_sample(x_clean_batch, t_batch, actual_noise_added)

    with tf.GradientTape() as tape:
        # Predict noise using our manual model
        predicted_noise = manual_denoise_model_forward(x_t_noisy, t_batch)
        loss = manual_mse_loss(actual_noise_added, predicted_noise)

    # Get gradients
    gradients = tape.gradient(loss, model_params)
    # Apply gradients manually
    manual_sgd_step(model_params, gradients, learning_rate_q2)

    return loss
```

```
#Full Training Loop
EPOCHS_Q2 = 20
training_losses_q2 = []

print(f"\nStarting training for {EPOCHS_Q2} epochs...")
for epoch in range(EPOCHS_Q2):
    epoch_start_time = time.time()
    total_epoch_loss = 0.0
    num_batches = 0

    pbar = tqdm(train_dataset_q2, desc=f"Epoch {epoch+1}/{EPOCHS_Q2}")
    for batch_clean_images in pbar:
        loss = train_step_manual_model(batch_clean_images)
        total_epoch_loss += loss.numpy()
        num_batches += 1
        pbar.set_postfix({"Loss": f"{loss.numpy():.4f}"})

    avg_epoch_loss = total_epoch_loss / num_batches
    training_losses_q2.append(avg_epoch_loss)
    epoch_end_time = time.time()
    print(f"Epoch [{epoch+1}/{EPOCHS_Q2}] Avg Loss: {avg_epoch_loss:.4f}, Time: {epoch_end_time - epoch_start_time:.2f}s")

print("Training complete.")
```

```
Starting training for 20 epochs...
Epoch 1/20: 100%|██████████| 469/469 [00:05<00:00, 91.89it/s, Loss=1.0172]
Epoch [1/20] Avg Loss: 1.0152, Time: 5.11s
Epoch 2/20: 100%|██████████| 469/469 [00:01<00:00, 322.27it/s, Loss=1.0106]
Epoch [2/20] Avg Loss: 1.0131, Time: 1.46s
Epoch 3/20: 100%|██████████| 469/469 [00:01<00:00, 335.30it/s, Loss=1.0083]
Epoch [3/20] Avg Loss: 1.0115, Time: 1.40s
Epoch 4/20: 100%|██████████| 469/469 [00:02<00:00, 184.41it/s, Loss=1.0137]
Epoch [4/20] Avg Loss: 1.0101, Time: 2.55s
Epoch 5/20: 100%|██████████| 469/469 [00:02<00:00, 184.04it/s, Loss=1.0128]
Epoch [5/20] Avg Loss: 1.0089, Time: 2.55s
Epoch 6/20: 100%|██████████| 469/469 [00:01<00:00, 324.52it/s, Loss=1.0066]
Epoch [6/20] Avg Loss: 1.0088, Time: 1.45s
Epoch 7/20: 100%|██████████| 469/469 [00:02<00:00, 184.39it/s, Loss=1.0102]
Epoch [7/20] Avg Loss: 1.0079, Time: 2.55s
Epoch 8/20: 100%|██████████| 469/469 [00:01<00:00, 334.90it/s, Loss=1.0032]
Epoch [8/20] Avg Loss: 1.0072, Time: 1.40s
Epoch 9/20: 100%|██████████| 469/469 [00:02<00:00, 184.37it/s, Loss=1.0010]
Epoch [9/20] Avg Loss: 1.0068, Time: 2.55s
Epoch 10/20: 100%|██████████| 469/469 [00:02<00:00, 184.38it/s, Loss=1.0133]
Epoch [10/20] Avg Loss: 1.0061, Time: 2.55s
Epoch 11/20: 100%|██████████| 469/469 [00:02<00:00, 184.37it/s, Loss=1.0148]
Epoch [11/20] Avg Loss: 1.0058, Time: 2.55s
Epoch 12/20: 100%|██████████| 469/469 [00:02<00:00, 184.45it/s, Loss=1.0068]
Epoch [12/20] Avg Loss: 1.0056, Time: 2.55s
Epoch 13/20: 100%|██████████| 469/469 [00:02<00:00, 184.25it/s, Loss=1.0072]
Epoch [13/20] Avg Loss: 1.0050, Time: 2.55s
Epoch 14/20: 100%|██████████| 469/469 [00:02<00:00, 184.37it/s, Loss=0.9961]
Epoch [14/20] Avg Loss: 1.0052, Time: 2.55s
Epoch 15/20: 100%|██████████| 469/469 [00:02<00:00, 184.42it/s, Loss=1.0113]
Epoch [15/20] Avg Loss: 1.0050, Time: 2.55s
Epoch 16/20: 100%|██████████| 469/469 [00:02<00:00, 184.25it/s, Loss=1.0030]
Epoch [16/20] Avg Loss: 1.0049, Time: 2.55s
Epoch 17/20: 100%|██████████| 469/469 [00:02<00:00, 183.92it/s, Loss=0.9921]
Epoch [17/20] Avg Loss: 1.0042, Time: 2.55s
Epoch 18/20: 100%|██████████| 469/469 [00:02<00:00, 184.14it/s, Loss=1.0083]
Epoch [18/20] Avg Loss: 1.0043, Time: 2.55s
Epoch 19/20: 100%|██████████| 469/469 [00:02<00:00, 184.38it/s, Loss=1.0025]
Epoch [19/20] Avg Loss: 1.0037, Time: 2.55s
Epoch 20/20: 100%|██████████| 469/469 [00:01<00:00, 264.68it/s, Loss=0.9960]Epoch [20/20] Avg Loss: 1.0041, Time: 1.78s
Training complete.
```

```
# Plotting training loss curve
plt.figure(figsize=(8, 5))
plt.plot(training_losses_q2, label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error Loss')
plt.title('Training Loss Curve (Manual Denoising Model)')
plt.legend()
plt.grid(True)
plt.show()
```




```
# displaying the sample outputs of denoising at step t = [200 -> 0]
def sample_manual_model(n_samples=16):
    print("\nStarting sampling (reverse diffusion)...")
    current_images = tf.random.normal((n_samples, img_height, img_width, channels))
    images_at_timesteps = []

    for t_idx in tqdm(reversed(range(T)), desc="Sampling Timesteps"):
        timesteps_for_model = tf.constant([t_idx] * n_samples, dtype=tf.int32)
        predicted_noise = manual_denoise_model_forward(current_images, timesteps_for_model)

        alpha_t = alpha[t_idx]
        alpha_hat_t = alpha_hat[t_idx]
        beta_t = beta[t_idx]

        coeff1 = 1.0 / tf.sqrt(alpha_t)
        coeff2 = (1.0 - alpha_t) / tf.sqrt(1.0 - alpha_hat_t)

        z = tf.random.normal(shape=tf.shape(current_images)) if t_idx > 0 else tf.zeros_like(current_images)
        current_images = coeff1 * (current_images - coeff2 * predicted_noise) + tf.sqrt(beta_t) * z

        if t_idx == T-1 or t_idx == int(T*0.75) or t_idx == int(T*0.5) or \
           t_idx == int(T*0.25) or t_idx == 0:
            images_at_timesteps.append(current_images)

    final_images_scaled = (images_at_timesteps[-1] + 1.0) / 2.0
    final_images_scaled = tf.clip_by_value(final_images_scaled, 0.0, 1.0)
    images_at_timesteps_scaled = [tf.clip_by_value((img + 1.0) / 2.0, 0.0, 1.0) for img in images_at_timesteps]
    return final_images_scaled, images_at_timesteps_scaled
```

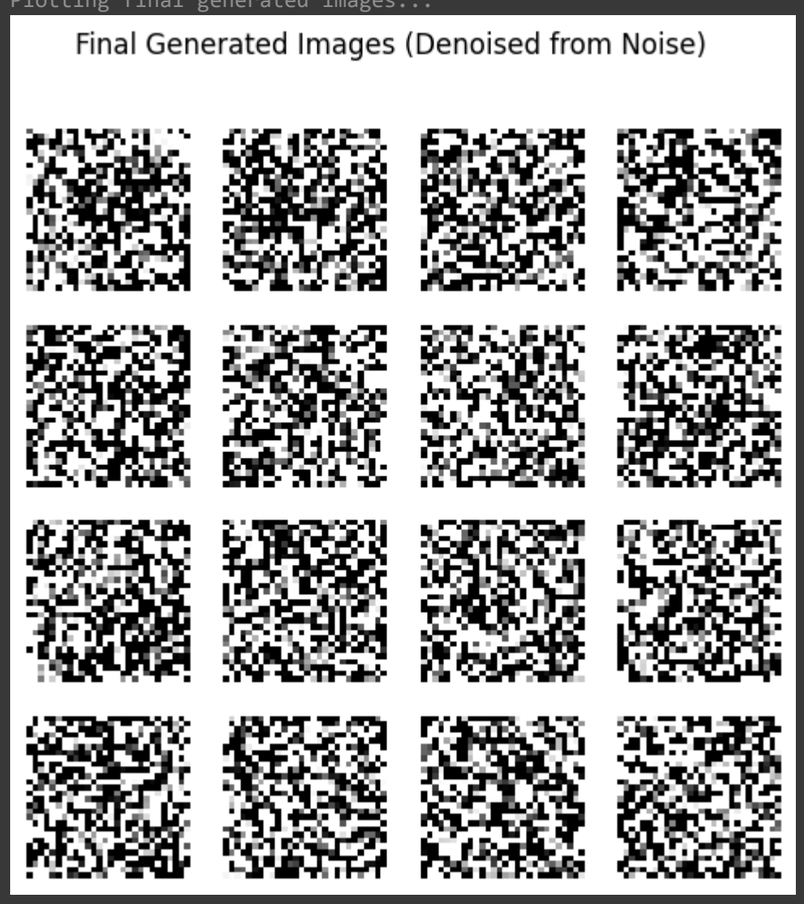
```
num_generated_samples = 16
```

```
generated_final_images, generated_intermediate_images = sample_manual_model(n_samples=num_generated_samples)
```

```
Starting sampling (reverse diffusion)...
Sampling Timesteps: 200it [00:01, 126.70it/s]
```

```
print("\nPlotting final generated images...")
plt.figure(figsize=(6, 6))
for i in range(num_generated_samples):
    plt.subplot(4, 4, i + 1)
    plt.imshow(generated_final_images[i, :, :, 0], cmap='gray')
    plt.axis('off')
plt.suptitle("Final Generated Images (Denoised from Noise)")
plt.show()
```

```
Plotting final generated images...
```




```
print("\nPlotting denoising process at different timesteps...")
num_snapshots = len(generated_intermediate_images)
num_examples_to_show = 4
plt.figure(figsize=(num_snapshots * 2, num_examples_to_show * 2))
snapshot_timesteps_display = [T, int(T*0.75)+1, int(T*0.5)+1, int(T*0.25)+1, 1]

for i in range(num_examples_to_show):
    for j in range(num_snapshots):
        plt.subplot(num_examples_to_show, num_snapshots, i * num_snapshots + j + 1)
        plt.imshow(generated_intermediate_images[j][i, :, :, 0], cmap='gray')
        plt.axis('off')
```

```
        if i == 0:
            plt.title(f"t={snapshot_timesteps_display[j]}")
plt.suptitle("Denoising Process (Selected Images from t=200 down to t=0)")
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

```
Plotting denoising process at different timesteps...
```

3. Class-Conditional Diffusion

```
# Diffusion Parameters (from Q1)
T = 200  # Total diffusion steps
beta_start = 1e-4
beta_end = 0.02
beta = tf.linspace(beta_start, beta_end, T)
beta = tf.cast(beta, dtype=tf.float32)
alpha = 1.0 - beta
alpha_hat = tf.math.cumprod(alpha)
num_classes = 10
```

```
# Loading and preprocess MNIST data (with labels)
print("\nLoading and preprocessing MNIST data with labels...")
(x_train, y_train_labels), (_, _) = tf.keras.datasets.mnist.load_data()
# Scale images to [-1, 1]
x_train = (x_train.astype(np.float32) / 255.0) * 2.0 - 1.0
x_train = np.expand_dims(x_train, axis=-1)
```

```
Loading and preprocessing MNIST data with labels...
```

```
#Condition on MNIST digit label (one-hot encode labels)
y_train_one_hot = to_categorical(y_train_labels, num_classes=num_classes)
y_train_one_hot = tf.cast(y_train_one_hot, dtype=tf.float32)
```

```
print(f"x_train shape: {x_train.shape}")
print(f"y_train_labels shape: {y_train_labels.shape}")
print(f"y_train_one_hot shape: {y_train_one_hot.shape}")
```

```
x_train shape: (60000, 28, 28, 1)
y_train_labels shape: (60000,)
y_train_one_hot shape: (60000, 10)
```

```
BATCH_SIZE_Q3 = 128
# Creating dataset of (image, one_hot_label) pairs
train_dataset_q3 = tf.data.Dataset.from_tensor_slices((x_train, y_train_one_hot))
train_dataset_q3 = train_dataset_q3.shuffle(10000).batch(BATCH_SIZE_Q3)
print("MNIST dataset with one-hot labels prepared.")
```

```
MNIST dataset with one-hot labels prepared.
```

```
def get_conditional_model(time_steps=T, num_cls=num_classes, img_shape=(28,28,1)):
    noisy_image_input = layers.Input(shape=img_shape, name="noisy_image_input")
    timestep_input = layers.Input(shape=(1,), name="timestep_input", dtype=tf.int32)
    label_input = layers.Input(shape=(num_cls,), name="label_input") # One-hot encoded label

    # Time embedding
    t_emb_dim = 32
    t_emb = layers.Embedding(time_steps, t_emb_dim, name="time_embedding")(timestep_input)
    t_emb = layers.Dense(img_shape[0] * img_shape[1], name="time_dense_project")(t_emb)
    t_emb = layers.Reshape((img_shape[0], img_shape[1], 1), name="time_reshape")(t_emb)

    # Label embedding
    label_emb_dim = 32
    l_emb = layers.Dense(label_emb_dim, activation='relu', name="label_dense_emb")(label_input)
    l_emb = layers.Dense(img_shape[0] * img_shape[1], name="label_dense_project")(l_emb)
    l_emb = layers.Reshape((img_shape[0], img_shape[1], 1), name="label_reshape")(l_emb)

    # Concatenate inputs: noisy image, time embedding, label embedding
    merged_input = layers.Concatenate(name="concat_inputs")([noisy_image_input, t_emb, l_emb])

    # U-Net like convolutional layers
```

```
    x = layers.Conv2D(32, 3, padding='same', activation='relu', name="conv1")(merged_input)
    x = layers.Conv2D(64, 3, padding='same', activation='relu', name="conv2")(x)
    x = layers.Conv2D(64, 3, padding='same', activation='relu', name="conv3")(x)
    x = layers.Conv2D(32, 3, padding='same', activation='relu', name="conv4")(x)
    # Output layer predicts the noise (1 channel)
    predicted_noise_output = layers.Conv2D(img_shape[-1], 3, padding='same', name="output_conv")(x)

    return tf.keras.Model([noisy_image_input, timestep_input, label_input], predicted_noise_output)
```

```
conditional_model = get_conditional_model()
print("\nConditional Denoising Model Summary:")
conditional_model.summary()
```

```
Conditional Denoising Model Summary:
Model: "functional"
```

Layer (type)	Output Shape	Param #	Connected to
timestep_input (InputLayer)	(None, 1)	0	-
label_input (InputLayer)	(None, 10)	0	-
time_embedding (Embedding)	(None, 1, 32)	6,400	timestep_input[0…
label_dense_emb (Dense)	(None, 32)	352	label_input[0][0]
time_dense_project (Dense)	(None, 1, 784)	25,872	time_embedding[0…
label_dense_project (Dense)	(None, 784)	25,872	label_dense_emb[…
noisy_image_input (InputLayer)	(None, 28, 28, 1)	0	-
time_reshape (Reshape)	(None, 28, 28, 1)	0	time_dense_proje…
label_reshape (Reshape)	(None, 28, 28, 1)	0	label_dense_proj…
concat_inputs (Concatenate)	(None, 28, 28, 3)	0	noisy_image_inpu… time_reshape[0][… label_reshape[0]…
conv1 (Conv2D)	(None, 28, 28, 32)	896	concat_inputs[0]…
conv2 (Conv2D)	(None, 28, 28, 64)	18,496	conv1[0][0]
conv3 (Conv2D)	(None, 28, 28, 64)	36,928	conv2[0][0]
conv4 (Conv2D)	(None, 28, 28, 32)	18,464	conv3[0][0]
output_conv (Conv2D)	(None, 28, 28, 1)	289	conv4[0][0]

```
Total params: 133,569 (521.75 KB)
```

```
# Optimizer & Loss function
optimizer_q3 = tf.keras.optimizers.Adam(learning_rate=1e-3)
mse_loss_q3 = tf.keras.losses.MeanSquaredError()
```

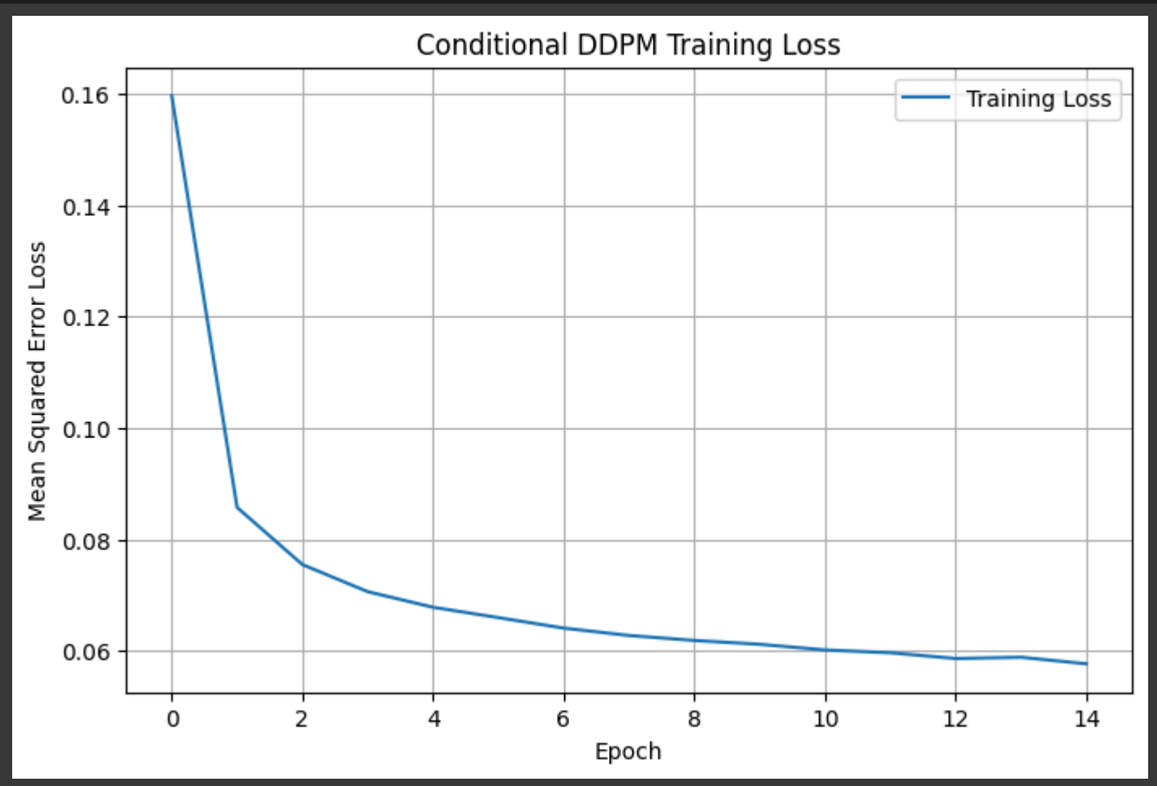
```
# Training Step for Conditional Model
@tf.function
def train_step_conditional(images_batch, labels_one_hot_batch):
    batch_s = tf.shape(images_batch)[0]
    # Sample random timesteps
    t_batch = tf.random.uniform(minval=0, maxval=T, shape=[batch_s], dtype=tf.int32)
    # Generate noise
    actual_noise_to_predict = tf.random.normal(shape=tf.shape(images_batch))
    # Create noisy images using q_sample
    x_t_noisy_batch = q_sample(images_batch, t_batch, actual_noise_to_predict)

    with tf.GradientTape() as tape:
        # Predict noise using the conditional model
        predicted_noise = conditional_model([x_t_noisy_batch, t_batch, labels_one_hot_batch])
        loss = mse_loss_q3(actual_noise_to_predict, predicted_noise)

    # Get gradients and apply them
    gradients = tape.gradient(loss, conditional_model.trainable_variables)
    optimizer_q3.apply_gradients(zip(gradients, conditional_model.trainable_variables))
    return loss
```

```
 #Full Training Loop for Conditional Model
EPOCHS_Q3 = 15
training_losses_q3 = []

print(f"\nStarting conditional model training for {EPOCHS_Q3} epochs...")
for epoch in range(EPOCHS_Q3):
    epoch_start_time = time.time()
    total_epoch_loss = 0.0
    num_batches_processed = 0

    pbar = tqdm(train_dataset_q3, desc=f"Epoch {epoch+1}/{EPOCHS_Q3}")
    for images_batch, labels_batch in pbar: # Now dataset yields (image, label)
        loss = train_step_conditional(images_batch, labels_batch)
        total_epoch_loss += loss.numpy()
        num_batches_processed += 1
        pbar.set_postfix({"Loss": f"{loss.numpy():.4f}"})

    avg_epoch_loss = total_epoch_loss / num_batches_processed
    training_losses_q3.append(avg_epoch_loss)
    epoch_end_time = time.time()
    print(f"Epoch [{epoch+1}/{EPOCHS_Q3}] Avg Loss: {avg_epoch_loss:.4f}, Time: {epoch_end_time - epoch_start_time:.2f}s")

print("Conditional model training complete.")
```

```
Starting conditional model training for 15 epochs...
Epoch 1/15: 100%|██████████| 469/469 [00:15<00:00, 30.90it/s, Loss=0.0860]
Epoch [1/15] Avg Loss: 0.1596, Time: 15.18s
Epoch 2/15: 100%|██████████| 469/469 [00:10<00:00, 45.87it/s, Loss=0.0827]
Epoch [2/15] Avg Loss: 0.0858, Time: 10.23s
```

```
Epoch 3/15: 100%|██████████| 469/469 [00:09<00:00, 49.44it/s, Loss=0.0756]
Epoch [3/15] Avg Loss: 0.0755, Time: 9.49s
Epoch 4/15: 100%|██████████| 469/469 [00:10<00:00, 45.87it/s, Loss=0.0685]
Epoch [4/15] Avg Loss: 0.0706, Time: 10.23s
Epoch 5/15: 100%|██████████| 469/469 [00:08<00:00, 53.03it/s, Loss=0.0759]
Epoch [5/15] Avg Loss: 0.0678, Time: 8.85s
Epoch 6/15: 100%|██████████| 469/469 [00:09<00:00, 48.05it/s, Loss=0.0800]
Epoch [6/15] Avg Loss: 0.0660, Time: 9.76s
Epoch 7/15: 100%|██████████| 469/469 [00:10<00:00, 45.87it/s, Loss=0.0588]
Epoch [7/15] Avg Loss: 0.0641, Time: 10.23s
Epoch 8/15: 100%|██████████| 469/469 [00:10<00:00, 45.87it/s, Loss=0.0644]
Epoch [8/15] Avg Loss: 0.0628, Time: 10.23s
Epoch 9/15: 100%|██████████| 469/469 [00:10<00:00, 45.88it/s, Loss=0.0527]
Epoch [9/15] Avg Loss: 0.0619, Time: 10.23s
Epoch 10/15: 100%|██████████| 469/469 [00:10<00:00, 45.87it/s, Loss=0.0575]
Epoch [10/15] Avg Loss: 0.0612, Time: 10.23s
Epoch 11/15: 100%|██████████| 469/469 [00:10<00:00, 46.50it/s, Loss=0.0598]
Epoch [11/15] Avg Loss: 0.0602, Time: 10.09s
Epoch 12/15: 100%|██████████| 469/469 [00:09<00:00, 48.54it/s, Loss=0.0642]
Epoch [12/15] Avg Loss: 0.0597, Time: 9.67s
Epoch 13/15: 100%|██████████| 469/469 [00:10<00:00, 45.88it/s, Loss=0.0617]
Epoch [13/15] Avg Loss: 0.0586, Time: 10.23s
Epoch 14/15: 100%|██████████| 469/469 [00:10<00:00, 45.88it/s, Loss=0.0588]
Epoch [14/15] Avg Loss: 0.0589, Time: 10.23s
Epoch 15/15: 100%|██████████| 469/469 [00:10<00:00, 45.88it/s, Loss=0.0524]Epoch [15/15] Avg Loss: 0.0577, Time: 10.23s
Conditional model training complete.
```

```
# Plotting training loss
plt.figure(figsize=(8,5))
plt.plot(training_losses_q3, label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error Loss')
plt.title('Conditional DDPM Training Loss')
plt.legend()
plt.grid(True)
plt.show()
```




```
# displaying denoised generations conditioned on classes [0, 1, 2]
# Modified Sampling Function for Conditional Model
def sample_conditional(model_to_sample, target_class_labels_int, n_samples_per_class=5):
    print(f"\nStarting conditional sampling for classes: {target_class_labels_int}...")
    all_generated_images = []

    for class_label_int in target_class_labels_int:
        print(f"  Generating samples for class: {class_label_int}")
        # Creating one-hot encoded label for this class
        label_one_hot = to_categorical([class_label_int] * n_samples_per_class, num_classes=num_classes)
        label_one_hot_tf = tf.cast(label_one_hot, dtype=tf.float32)

        # Starting with pure random noise
        current_images = tf.random.normal((n_samples_per_class, 28, 28, 1))

        for t_idx in tqdm(reversed(range(T)), desc=f"Sampling Class {class_label_int}", leave=False):
            timesteps_for_model = tf.constant([t_idx] * n_samples_per_class, dtype=tf.int32)

            predicted_noise = model_to_sample([current_images, timesteps_for_model, label_one_hot_tf])

            alpha_t = alpha[t_idx]
            alpha_hat_t = alpha_hat[t_idx]
            beta_t = beta[t_idx]

            coeff1 = 1.0 / tf.sqrt(alpha_t)
            coeff2 = (1.0 - alpha_t) / tf.sqrt(1.0 - alpha_hat_t)

            z = tf.random.normal(shape=tf.shape(current_images)) if t_idx > 0 else tf.zeros_like(current_images)
            current_images = coeff1 * (current_images - coeff2 * predicted_noise) + tf.sqrt(beta_t) * z

        # Scaling images from [-1, 1] to [0, 1] for plotting
        final_images_scaled = (current_images + 1.0) / 2.0
        final_images_scaled = tf.clip_by_value(final_images_scaled, 0.0, 1.0)
        all_generated_images.append(final_images_scaled)

    return all_generated_images
```

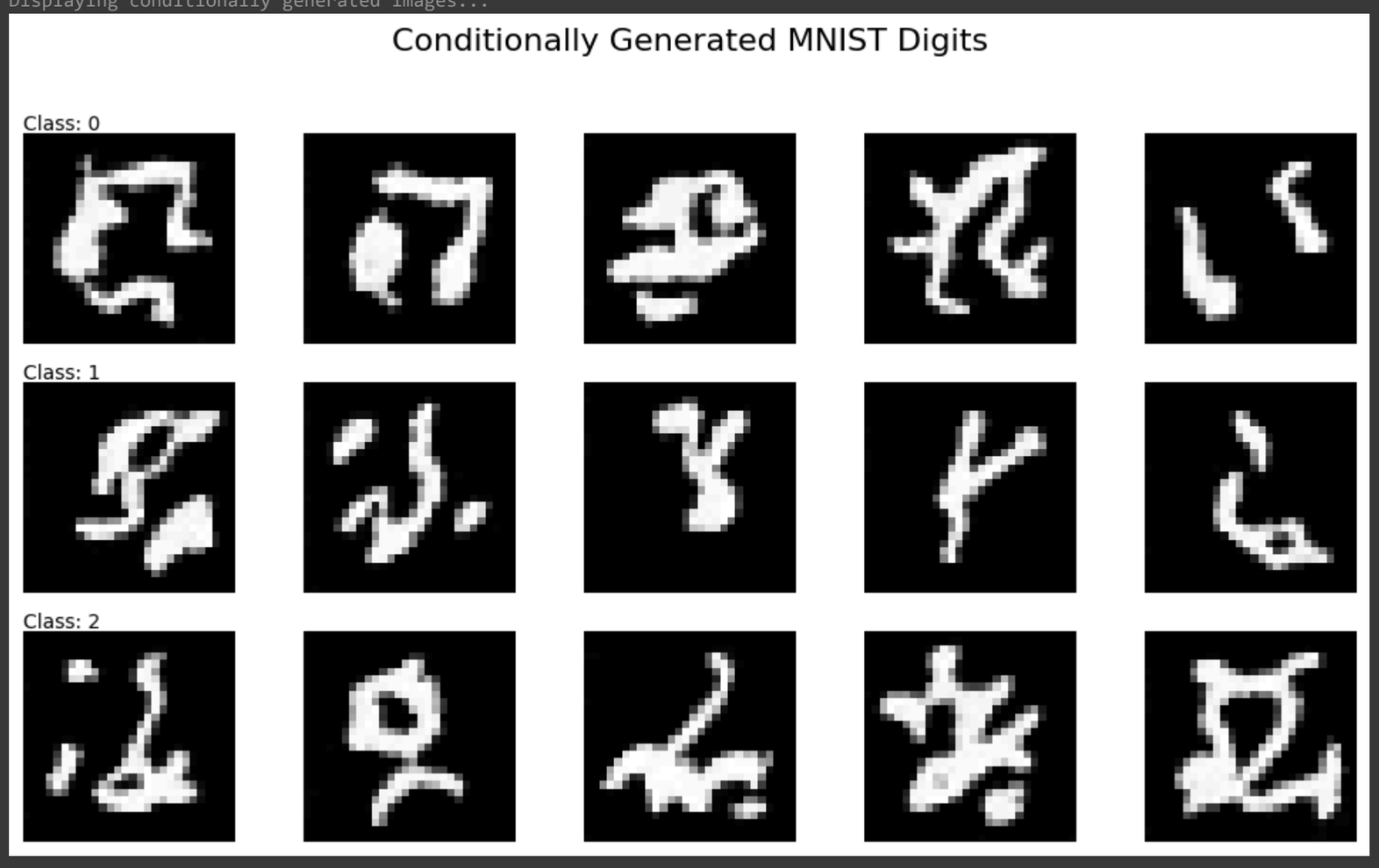
```
# Generating images conditioned on classes 0, 1, 2
classes_to_generate = [0, 1, 2]
num_examples_per_class = 5 # Showing 5 examples for each class
conditional_generated_images_list = sample_conditional(conditional_model, classes_to_generate, n_samples_per_class=num_examples_per_class)
```

```
Starting conditional sampling for classes: [0, 1, 2]...
  Generating samples for class: 0
  Generating samples for class: 1
  Generating samples for class: 2
```

```
print("\nDisplaying conditionally generated images...")
plt.figure(figsize=(num_examples_per_class * 2, len(classes_to_generate) * 2))
for i, class_label_int in enumerate(classes_to_generate):
    images_for_this_class = conditional_generated_images_list[i]
    for j in range(num_examples_per_class):
        plt.subplot(len(classes_to_generate), num_examples_per_class, i * num_examples_per_class + j + 1)
```

```
        plt.imshow(images_for_this_class[j, :, :, 0], cmap='gray')
        plt.axis('off')
        if j == 0:
            plt.title(f"Class: {class_label_int}", loc='left', fontsize=10, pad=-10)
plt.suptitle("Conditionally Generated MNIST Digits", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

```
Displaying conditionally generated images...
```



4. Text-to-Image with Pretrained Stable Diffusion

```
# Loading a basic Huggingface pipeline of Stable Diffusion.
```

```
import torch
from diffusers import StableDiffusionPipeline
from PIL import Image
```

```
model_id_sd = "CompVis/stable-diffusion-v1-4"
```

```
device_sd = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device_sd}")
```

```
Using device: cuda
```

```
if device_sd.type == 'cuda':
    try:
        pipe_sd = StableDiffusionPipeline.from_pretrained(
            model_id_sd,
            torch_dtype=torch.float16,
            use_safetensors=True
        )
    except Exception as e:
        print(f"Warning: Could not load with float16/safetensors ({e}).")
        pipe_sd = StableDiffusionPipeline.from_pretrained(model_id_sd)
else:
    pipe_sd = StableDiffusionPipeline.from_pretrained(model_id_sd, use_safetensors=True)
```

```
Fetching 16 files: 100%                                    16/16 [01:43<00:00,  7.66s/it]
text_encoder/model.safetensors: 100%                       492M/492M [00:43<00:00, 11.0MB/s]
vae/diffusion_pytorch_model.safetensors: 100%              335M/335M [00:52<00:00, 8.78MB/s]
unet/diffusion_pytorch_model.safetensors: 100%             3.44G/3.44G [01:43<00:00, 113MB/s]
safety_checker/model.safetensors: 100%                     1.22G/1.22G [00:37<00:00, 32.5MB/s]
Loading pipeline components...: 100%                       7/7 [00:27<00:00,  2.97s/it]
```

```
pipe_sd = pipe_sd.to(device_sd)
print("Stable Diffusion pipeline loaded and moved to device.")
```

```
Stable Diffusion pipeline loaded and moved to device.
```

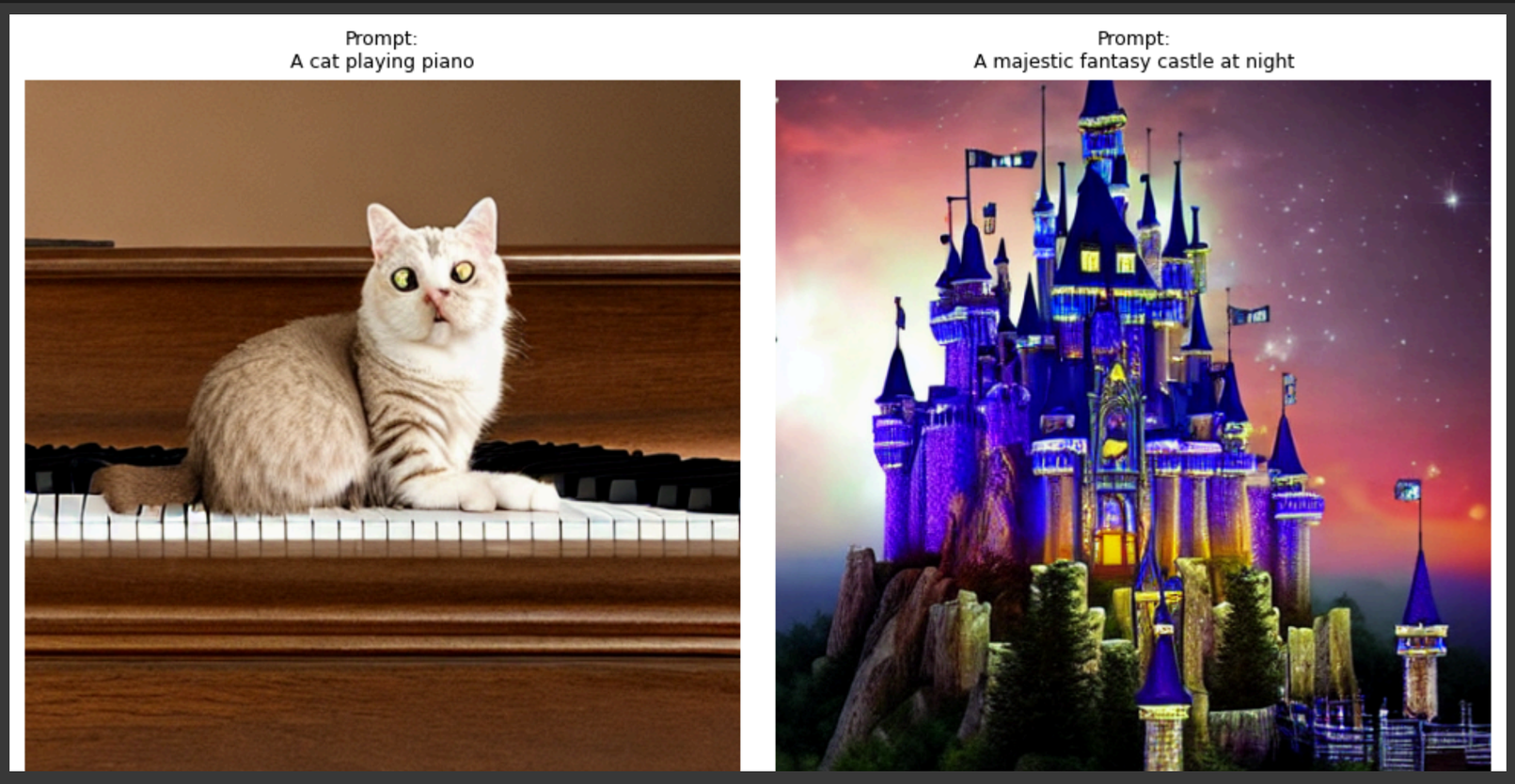
```
# Define prompts
prompts_sd = [
    "A cat playing piano",
    "A majestic fantasy castle at night"
]
print(f"\nDefined {len(prompts_sd)} prompts for image generation.")
```

```
Defined 2 prompts for image generation.
```

```
# Generate images and display them.
# Parameters for generation
num_inference_steps_sd = 30  # Fewer steps for faster generation
guidance_scale_sd = 7.5      # How much the generation should stick to the prompt
```

```
generated_images_sd = []
print("\nGenerating images...")

for i, current_prompt in enumerate(prompts_sd):
    print(f"  Generating image for prompt {i+1}/{len(prompts_sd)}: \"{current_prompt[:70]}...\"")
    generator_sd = torch.Generator(device=device_sd).manual_seed(42 + i*100)

    # Generate image
    with torch.autocast(device_sd.type if device_sd.type == "cuda" else "cpu"):
```

```
    image_output_sd = pipe_sd(
        current_prompt,
        num_inference_steps=num_inference_steps_sd,
        guidance_scale=guidance_scale_sd,
        generator=generator_sd
    ).images[0]

generated_images_sd.append(image_output_sd)
print(f"  Image {i+1} generated.")
# Display the generated images
print("\nDisplaying generated images...")
num_images_to_display = len(generated_images_sd)
cols_display = min(num_images_to_display, 3) # Max 3 images per row
rows_display = (num_images_to_display + cols_display - 1)
```

```
Generating images...
  Generating image for prompt 1/2: "A cat playing piano..."
100% [                                  ] 30/30 [00:04<00:00,  6.73it/s]
  Image 1 generated.

Displaying generated images...
  Generating image for prompt 2/2: "A majestic fantasy castle at night..."
100% [                                  ] 30/30 [00:04<00:00,  6.67it/s]
  Image 2 generated.

Displaying generated images...
```

```
plt.figure(figsize=(cols_display * 5, rows_display * 5))
for i, img_sd in enumerate(generated_images_sd):
    plt.subplot(rows_display, cols_display, i + 1)
    plt.imshow(img_sd)
    title_prompt = prompts_sd[i]
    if len(title_prompt) > 60:
        title_prompt = title_prompt[:57] + "..."
    plt.title(f"Prompt:\n{title_prompt}", fontsize=9)
    plt.axis('off')
plt.tight_layout()
plt.show()
```